Software Requirements Specification

for

SkipTheQueue

AI-Powered Smart Queue Management and Appointment Scheduling System

Version 1.0

Prepared by: Amruta Gaikwad (24BPS1064)

Vellore Institute of Technology Chennai

Date: 18th July 2026

Revision History

Name	Date	Reason for Changes	Version
<Author>	<Date>	Initial draft of SRS	1.0

Table of Contents

Revision History 2

1. Introduction 5

1.1 Purpose 5

1.2 Document Conventions 5

1.3 Intended Audience and Reading Suggestions 5

1.4 Product Scope 5

1.5 References 5

2. Overall Description 6

2.1 Product Perspective 6

2.2 Product Functions 6

2.3 User Classes and Characteristics 6

2.4 Operating Environment 6

2.5 Design and Implementation Constraints 7

2.6 User Documentation 7

2.7 Assumptions and Dependencies 7

3. External Interface Requirements 8

3.1 User Interfaces 8

3.2 Hardware Interfaces 8

3.3 Software Interfaces 8

3.4 Communications Interfaces 8

4. System Features 9

4.1 User Registration and Authentication 9

4.2 Smart Appointment Booking 9

4.3 Service Time Prediction (Machine Learning) 10

4.4 Real-Time Queue Tracking 10

4.5 Dynamic Queue Optimization 11

4.6 Priority Management 11

4.7 Nearby Low-Crowd Branch Recommendation 12

4.8 Notification System 12

4.9 Staff Workload and Manager Dashboard 13

4.10 Demand Forecasting 13

4.11 QR-Based Contactless Check-In 14

4.12 Feedback and Service Rating 14

4.13 AI-Assisted Complaint and Grievance Management 15

4.14 Multilingual and Voice Assistant Support 15

4.15 Offline / SMS-Based Booking for Non-Smartphone Users 16

5. Other Nonfunctional Requirements 17

5.1 Performance Requirements 17

5.2 Safety Requirements 17

5.3 Security Requirements 17
5.4 Software Quality Attributes 17
5.5 Business Rules 18
6. Other Requirements 19
6.1 Database Requirements 19
6.2 Internationalization Requirements 19
6.3 Legal and Compliance Requirements 19
Appendix A: Glossary 20
Appendix B: Analysis Models 21
Appendix C: To Be Determined List 22

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for SkipTheQueue, Version 1.0 — an AI-powered smart queue management and appointment scheduling platform. It covers the complete system: citizen-facing web and mobile applications, the backend scheduling and prediction engine, and the administrative/staff dashboard. This document is intended to guide design, development, and testing, and to serve as the basis of agreement between stakeholders on what the system will and will not do.

1.2 Document Conventions

Each functional requirement is uniquely tagged with a sequence number in the format REQ-N and is numbered continuously across all system features for traceability. Requirement statements use "shall" to denote mandatory behavior. Priorities (High, Medium, Low) are assigned at the feature level; a High-priority feature's requirements are treated as High unless individually noted otherwise. This document follows the IEEE-recommended SRS template (adapted from Karl Wiegers' SRS format).

1.3 Intended Audience and Reading Suggestions

This document is intended for the project's software developers, project managers, quality assurance/testers, UI/UX designers, and evaluating faculty/stakeholders. Developers and testers should focus on Sections 3–5 for interface and functional detail. Project managers and evaluators may begin with Sections 1–2 for scope and context before reviewing feature-level detail in Section 4.

1.4 Product Scope

SkipTheQueue is a centralized digital platform that replaces first-come-first-served token queues with intelligent appointment scheduling. Citizens can book appointments at public service centers (banks, Aadhaar enrollment centers, ration shops, passport offices, municipal offices, hospitals, electricity boards, and post offices), receive a machine-learning-predicted service duration, track their live queue position, and receive a recommendation to a less crowded nearby branch when applicable. Organizations gain a real-time operations dashboard, demand forecasting, and workload-balancing tools. The product's objective is to reduce citizen waiting time by an estimated 70–80%, improve staff and counter utilization, and make public services more inclusive and accessible for elderly, pregnant, and differently-abled citizens.

1.5 References

- Wiegers, K. E., "Software Requirements Specification Template," 1999 (base template structure).
- IEEE Recommended Practice for Software Requirements Specifications (IEEE 830 / ISO/IEC/IEEE 29148).
- SkipTheQueue — Project Concept Note (internal document, provided by project author).
- FastAPI, React.js, Flutter, and PostgreSQL official documentation (referenced for technical feasibility).

2. Overall Description

2.1 Product Perspective

SkipTheQueue is a new, self-contained platform rather than a follow-on to an existing product. It is designed to be deployed independently by individual service organizations (e.g., a bank branch network or a municipal corporation) or, longer term, integrated with government service portals. The system is composed of a citizen-facing Web/Mobile application, a FastAPI backend server, a machine learning engine, a mapping module, a relational database, and an administrative dashboard for managers and staff, as illustrated in Appendix B.

2.2 Product Functions

- Online appointment booking with AI-recommended time slots.
- Machine-learning-based service duration prediction.
- Real-time, live queue position and wait-time tracking.
- Dynamic queue re-optimization for no-shows and idle counters.
- Priority scheduling for senior citizens, pregnant women, and differently-abled users.
- Recommendation of nearby, less-crowded service branches.
- Multi-channel notifications (push, SMS, email).
- Staff workload dashboard and demand forecasting for administrators.
- QR-based contactless check-in.
- Feedback/rating collection and AI-assisted complaint management.
- Multilingual, voice-assisted, and SMS/IVR access for digital inclusion.

2.3 User Classes and Characteristics

User Class	Characteristics
Regular Citizen	General public booking routine services; expected to use the mobile/web app; moderate technical proficiency assumed.
Senior Citizen / Pregnant Woman / Differently-Abled User	Requires priority handling and accessible interfaces; may rely on SMS/IVR or assisted kiosk check-in rather than the app.
Staff / Counter Operator	Uses the counter interface to call the next customer, mark services complete, and view assigned appointments; frequent, daily use.
Branch Manager	Uses the dashboard to monitor utilization, reassign staff, and review forecasts; moderate frequency, high privilege.
System Administrator	Configures services, branches, priority rules, and manages user accounts; highest privilege level, infrequent but critical use.

2.4 Operating Environment

The citizen-facing application shall run as a responsive web application (modern browsers: Chrome, Firefox, Edge, Safari) and as a native mobile application for Android and iOS built with Flutter. The backend shall run on a Linux-based server environment (containerized deployment recommended) using FastAPI (Python 3.10+) and PostgreSQL. Real-time features shall use WebSockets, and the system shall be deployable on cloud/free-tier hosting suitable for pilot or student-project scale (e.g., Vercel, Render, Supabase), with a clear upgrade path to production-grade infrastructure.

2.5 Design and Implementation Constraints

- The system must use free/open-source components for the initial (pilot/academic) deployment as specified in Appendix B.
- Aadhaar-based verification, where used, must comply with applicable data protection and UIDAI guidelines; the system shall not store raw Aadhaar numbers unencrypted.
- The ML prediction engine must return results within 500 ms to avoid perceptible delay in the booking flow.
- The system must support at least 2 concurrent languages at launch (see Section 4.14).
- All communications must occur over encrypted channels (HTTPS/TLS; secure WebSocket).

2.6 User Documentation

The delivered product shall include an in-app onboarding walkthrough for citizens, a staff quick-reference guide for the counter interface, an administrator manual covering configuration and reporting, and contextual/inline help tooltips throughout the application. Documentation shall be made available in at least the languages supported per Section 4.14.

2.7 Assumptions and Dependencies

- It is assumed that participating service centers will provide accurate historical service-time data for ML model training.
- It is assumed that citizens booking via the app have access to a mobile number for OTP verification and notifications.
- The system depends on third-party services: SMS gateway, Firebase Cloud Messaging (push notifications), and OpenStreetMap/Leaflet.js (mapping); an outage in any of these will degrade, but not fully disable, core booking functionality.
- It is assumed that each partner organization designates at least one administrator responsible for initial service/branch configuration.

3. External Interface Requirements

3.1 User Interfaces

The citizen application shall provide: a service/branch search and selection screen, a slot-recommendation and booking confirmation screen, a live queue-status screen, a notification/history screen, and a profile/settings screen (including language and accessibility category). The staff interface shall provide a single-screen counter view showing the current and next customer, a "call next" action, and service-completion controls. The administrator dashboard shall provide summary metrics, staff/counter assignment controls, forecasting charts, and exportable reports. All interfaces shall conform to WCAG 2.1 AA accessibility guidelines, including sufficient color contrast, screen-reader labels, and scalable text.

3.2 Hardware Interfaces

The system shall support optional check-in kiosks equipped with a QR/barcode scanner and a display screen for on-site check-in at service branches. No other specialized hardware is required; the mobile application shall operate on standard Android and iOS smartphones with camera access for QR scanning.

3.3 Software Interfaces

Interface	Purpose
PostgreSQL (Supabase)	Primary relational data store for users, appointments, branches, and queue state.
FastAPI backend	REST/WebSocket API layer connecting the frontend, ML engine, and database.
ML Service (Scikit-learn, XGBoost, Prophet)	Service-duration prediction, no-show prediction, and demand forecasting.
Firebase Cloud Messaging	Delivery of push notifications to mobile devices.
SMTP / Gmail API	Delivery of email notifications and confirmations.
SMS Gateway	Delivery of SMS notifications and SMS-based booking (Section 4.15).
OpenStreetMap / Leaflet.js	Branch location mapping and crowd-density visualization.
Aadhaar/UIDAI verification API (optional)	Identity verification for services that require it, where legally permitted.

3.4 Communications Interfaces

All client-server communication shall use HTTPS (TLS 1.2 or higher). Real-time queue and notification updates shall use secure WebSocket connections (WSS). SMS communications shall use a documented short-code/keyword format described in the SMS Booking module (Section 4.15). Data exchanged with third-party services (SMS gateway, FCM, mapping) shall use their respective published REST APIs.

4. System Features

This section organizes functional requirements by system feature/module. Each feature includes a description and priority, representative stimulus/response sequences, and itemized functional requirements. Features 4.1–4.10 correspond to the originally proposed modules, elaborated with additional detail; Features 4.11–4.15 are additional modules proposed to extend the system's reach, inclusivity, and operational value.

4.1 User Registration and Authentication

4.1.1 Description and Priority

Allows citizens, staff, and administrators to create secure accounts and access the platform according to their role. Supports identity verification through Aadhaar or mobile OTP to prevent duplicate or fraudulent bookings.

Priority: High

4.1.2 Stimulus/Response Sequences

- User submits phone number → system sends OTP for verification.
- User completes profile with category (Regular, Senior Citizen, Pregnant Woman, Differently-Abled) → system stores classification for priority scheduling.
- Staff/Administrator logs in with organizational credentials → system grants role-based dashboard access.

4.1.3 Functional Requirements

REQ-1: The system shall allow a new user to register using a mobile number and OTP verification.

REQ-2: The system shall allow optional Aadhaar-based identity verification for government service centers that require it.

REQ-3: The system shall allow users to select an accessibility/priority category during profile creation and shall permit later updates by verified staff.

REQ-4: The system shall support three role types at minimum: Citizen, Staff, and Administrator, each with distinct permissions.

REQ-5: The system shall lock an account for 15 minutes after 5 consecutive failed login attempts.

REQ-6: The system shall allow users to reset a forgotten password or PIN via OTP.

4.2 Smart Appointment Booking

4.2.1 Description and Priority

Enables citizens to select a service center, service category, and preferred date/time, and receive a system-recommended slot based on predicted service duration and current load.

Priority: High

4.2.2 Stimulus/Response Sequences

- User selects a service type (e.g., Aadhaar Update, Cash Deposit, Ration Card Registration) → system displays estimated duration and available slots.

- User selects a preferred date/time → system either confirms the slot or suggests the nearest optimal alternative if the requested slot is overbooked.
- User confirms booking → system generates a digital token/QR code and appointment confirmation.

4.2.3 Functional Requirements

REQ-7: The system shall display a list of available services per selected service center.

REQ-8: The system shall recommend an optimal appointment slot using predicted service duration and existing bookings.

REQ-9: The system shall prevent double-booking of the same time slot at a given counter.

REQ-10: The system shall allow a user to reschedule or cancel an appointment up to a configurable cutoff time before the slot.

REQ-11: The system shall generate a unique appointment ID and a scannable QR code for every confirmed booking.

REQ-12: The system shall notify the user of booking confirmation within 5 seconds of submission.

4.3 Service Time Prediction (Machine Learning)

4.3.1 Description and Priority

Predicts the expected duration of a requested service using historical processing data, day-of-week trends, staff availability, and current crowd level, allowing more accurate slot allocation.

Priority: High

4.3.2 Stimulus/Response Sequences

- Booking module requests a duration estimate for a selected service → ML engine returns predicted minutes with a confidence indicator.
- Administrator uploads updated historical service logs → system retrains the prediction model on a scheduled basis.

4.3.3 Functional Requirements

REQ-13: The system shall predict service duration using at minimum: service type, historical average processing time, day of week, and current crowd level.

REQ-14: The system shall expose the predicted duration to the booking module via an internal API with a response time under 500 ms.

REQ-15: The system shall log actual vs. predicted duration for every completed service to support model retraining.

REQ-16: The system shall retrain the prediction model on a configurable schedule (e.g., weekly) without service downtime.

REQ-17: The system shall flag predictions with low confidence for administrator review.

4.4 Real-Time Queue Tracking

4.4.1 Description and Priority

Provides citizens with a live view of their queue position, updated expected wait time, and assigned counter, reducing uncertainty and the need to arrive early.

Priority: High

4.4.2 Stimulus/Response Sequences

- User opens the app during an active appointment → system displays live position, updated wait time, and counter assignment.
- Queue position changes due to no-shows or reordering → system pushes an updated notification to the affected users.

4.4.3 Functional Requirements

REQ-18: The system shall display the user's current queue position and estimated wait time, refreshed at least every 30 seconds.

REQ-19: The system shall notify a user when they are within 3 positions of being served.

REQ-20: The system shall display the assigned counter number once allocated.

REQ-21: The system shall maintain queue-state consistency across concurrent users using real-time updates (WebSockets).

4.5 Dynamic Queue Optimization

4.5.1 Description and Priority

Automatically adjusts the queue when a customer arrives late, is absent, or a counter becomes idle, moving the next eligible customer forward instead of leaving counters unused.

Priority: High

4.5.2 Stimulus/Response Sequences

- Scheduled appointment time passes without check-in → system marks the slot as delayed and moves the next customer forward after a grace period.
- A counter becomes idle → system automatically assigns the next customer in the optimized order.

4.5.3 Functional Requirements

REQ-22: The system shall apply a configurable grace period (default 10 minutes) before treating an appointment as a no-show.

REQ-23: The system shall automatically reassign an idle counter to the next eligible customer in the queue.

REQ-24: The system shall allow a late customer to rejoin the queue at the next available slot rather than losing the appointment entirely.

REQ-25: The system shall log all automatic queue reordering events for audit purposes.

4.6 Priority Management

4.6.1 Description and Priority

Ensures inclusive access by assigning priority scores to senior citizens, pregnant women, and differently-abled individuals so they are served with appropriate precedence without disrupting overall fairness.

Priority: High

4.6.2 Stimulus/Response Sequences

- A user with a priority classification checks in → system inserts them into the queue according to their priority score.
- Two or more priority users are waiting → system serves them in order of priority score and arrival time.

4.6.3 Functional Requirements

REQ-26: The system shall assign a priority score to each user category (e.g., Regular = 1, Senior Citizen = 3, Pregnant Woman = 4, Differently-Abled = 4).

REQ-27: The system shall insert priority users into the active queue without removing more than one regular-user slot per priority insertion, to preserve fairness.

REQ-28: The system shall allow administrators to configure priority weightings per organization/policy.

REQ-29: The system shall visibly flag priority appointments on the staff/counter dashboard.

4.7 Nearby Low-Crowd Branch Recommendation

4.7.1 Description and Priority

Analyzes crowd density and wait times across nearby service centers and recommends a less crowded alternative to the user before or during booking.

Priority: Medium

4.7.2 Stimulus/Response Sequences

- User searches for a service → system compares wait times across nearby branches and highlights the fastest option.
- Current branch becomes heavily congested → system suggests an alternative branch with shorter wait time within a configurable radius.

4.7.3 Functional Requirements

REQ-30: The system shall calculate and display current wait time for all branches offering the requested service within a configurable radius.

REQ-31: The system shall recommend the branch with the lowest combined (distance-adjusted) wait time.

REQ-32: The system shall display branch locations on an interactive map using OpenStreetMap/Leaflet.js.

REQ-33: The system shall allow the user to accept a recommended branch and rebook with a single action.

4.8 Notification System

4.8.1 Description and Priority

Keeps users informed at each stage of the appointment lifecycle through multiple channels, including booking confirmation, reminders, and real-time queue alerts.

Priority: High

4.8.2 Stimulus/Response Sequences

- Appointment is booked, modified, or cancelled → system sends a confirmation notification via the user's preferred channel.
- Appointment time approaches (configurable interval) → system sends a reminder notification with counter details.

4.8.3 Functional Requirements

REQ-34: The system shall support notifications via push notification, SMS, and email at minimum.

REQ-35: The system shall send a reminder notification 20 minutes (configurable) before the scheduled appointment.

REQ-36: The system shall allow users to select and update their preferred notification channel(s).

REQ-37: The system shall log delivery status for every notification sent.

4.9 Staff Workload and Manager Dashboard

4.9.1 Description and Priority

Provides administrators and managers with visibility into counter utilization, staff efficiency, and peak-hour patterns to support staffing decisions.

Priority: Medium

4.9.2 Stimulus/Response Sequences

- Manager opens the dashboard → system displays total customers served, peak hours, counter utilization, and staff efficiency metrics.
- Manager requests a staffing recommendation → system suggests counter/staff allocation based on forecasted demand.

4.9.3 Functional Requirements

REQ-38: The system shall display real-time counts of customers served, waiting, and no-shows per branch.

REQ-39: The system shall display counter utilization and average handling time per staff member.

REQ-40: The system shall allow managers to manually reassign staff to counters from the dashboard.

REQ-41: The system shall provide a downloadable (CSV/PDF) daily operations summary.

4.10 Demand Forecasting

4.10.1 Description and Priority

Uses historical data to forecast daily and weekly customer volume, including seasonal spikes such as festivals and salary-day rushes, to support proactive staffing.

Priority: Medium

4.10.2 Stimulus/Response Sequences

- System runs scheduled forecasting job → generates predicted demand for the next 7/30 days per branch.
- Forecast indicates an unusual spike (e.g., festival, salary day) → system alerts branch managers in advance.

4.10.3 Functional Requirements

REQ-42: The system shall forecast expected daily customer volume per branch for at least the next 7 days.

REQ-43: The system shall incorporate recognized public holidays and salary-cycle patterns into forecasts.

REQ-44: The system shall alert managers when forecasted demand exceeds available capacity by a configurable threshold.

REQ-45: The system shall allow administrators to review and override forecast-driven staffing suggestions.

4.11 QR-Based Contactless Check-In

4.11.1 Description and Priority

Allows citizens to check in for their appointment by scanning a QR code at a kiosk or with the mobile app, reducing manual token counter dependency and enabling faster verification.

Priority: Medium

4.11.2 Stimulus/Response Sequences

- User arrives at the branch and scans their appointment QR code at a kiosk or via the app → system marks the user as checked-in and updates the live queue.
- Staff scans the user's QR code manually → system verifies identity and appointment validity instantly.

4.11.3 Functional Requirements

REQ-46: The system shall generate a unique, tamper-resistant QR code for every confirmed appointment.

REQ-47: The system shall verify a scanned QR code against the appointment database within 2 seconds.

REQ-48: The system shall reject or flag QR codes that are expired, already used, or do not match the scheduled date.

REQ-49: The system shall support check-in via a kiosk application in addition to the mobile app.

4.12 Feedback and Service Rating

4.12.1 Description and Priority

Collects citizen feedback and star ratings after each completed service to measure satisfaction and identify centers or staff needing improvement.

Priority: Medium

4.12.2 Stimulus/Response Sequences

- Service is marked complete → system prompts the user for a rating and optional comment.
- Administrator opens the analytics dashboard → system displays aggregated ratings and flags low-rated branches/staff.

4.12.3 Functional Requirements

REQ-50: The system shall prompt users for a 1–5 star rating and optional text comment after each completed appointment.

REQ-51: The system shall aggregate ratings by branch, service type, and staff member.

REQ-52: The system shall flag any branch or staff member whose average rating falls below a configurable threshold for administrator review.

REQ-53: The system shall allow users to submit feedback anonymously if preferred.

4.13 AI-Assisted Complaint and Grievance Management

4.13.1 Description and Priority

Provides a structured channel for citizens to raise complaints, with AI-based categorization and routing to the appropriate department for faster resolution.

Priority: Low

4.13.2 Stimulus/Response Sequences

- User submits a complaint through the app → system categorizes it (e.g., delay, staff behavior, technical issue) and routes it to the responsible team.
- Complaint remains unresolved beyond the SLA window → system escalates it to a senior administrator.

4.13.3 Functional Requirements

REQ-54: The system shall allow users to submit complaints with category, description, and optional attachment.

REQ-55: The system shall use natural-language classification to auto-tag complaints by type.

REQ-56: The system shall track complaint status (Open, In Progress, Resolved) and notify the user of updates.

REQ-57: The system shall escalate complaints unresolved after a configurable SLA period (e.g., 72 hours).

4.14 Multilingual and Voice Assistant Support

4.14.1 Description and Priority

Improves accessibility for citizens across linguistic and literacy backgrounds by supporting regional languages and voice-guided navigation/booking.

Priority: Low

4.14.2 Stimulus/Response Sequences

- User selects a preferred regional language → system renders all screens and notifications in that language.
- User activates the voice assistant → system guides the user through booking using speech prompts and recognition.

4.14.3 Functional Requirements

REQ-58: The system shall support at least English and one configurable regional language at initial release, with an extensible translation framework.

REQ-59: The system shall allow users to change language preference at any time from account settings.

REQ-60: The system shall provide a voice-guided booking flow for at least the core steps: select service, select slot, confirm booking.

REQ-61: The system shall present all SMS and voice-based interactions in the user's selected language where supported.

4.15 Offline / SMS-Based Booking for Non-Smartphone Users

4.15.1 Description and Priority

Ensures digital inclusion for citizens without smartphones or reliable internet access by allowing appointment booking and status checks via SMS or a toll-free IVR line.

Priority: Medium

4.15.2 Stimulus/Response Sequences

- User sends a structured SMS (e.g., service code + branch code) to the designated number → system books the nearest available slot and replies with confirmation and token details.
- User calls the IVR helpline → system provides queue status and next available slot via automated voice prompts.

4.15.3 Functional Requirements

REQ-62: The system shall accept appointment requests via SMS using a documented short-code format.

REQ-63: The system shall reply to a valid SMS booking request with confirmation, date, time, and token number within 60 seconds.

REQ-64: The system shall provide an IVR/toll-free channel for checking appointment status without requiring app or internet access.

REQ-65: The system shall apply the same priority and grace-period rules to SMS/IVR bookings as to app bookings.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The system shall support at least 5,000 concurrent users per deployment instance without degradation of booking response time beyond 2 seconds.
- Live queue-position updates shall propagate to affected clients within 3 seconds of a queue-state change.
- The ML prediction service shall return a duration estimate within 500 ms for 95% of requests.
- The system shall maintain 99.5% uptime during service-center operating hours.

5.2 Safety Requirements

- The system shall never allow a priority-classified appointment (senior citizen, pregnant woman, differently-abled) to be silently dropped or overwritten by an automated reordering process without an audit log entry.
- In the event of backend or connectivity failure, the system shall preserve the last known valid queue order and require manual reconciliation before automated reordering resumes.

5.3 Security Requirements

- The system shall use JWT-based authentication for all API access, with token expiry and refresh mechanisms.
- Personally identifiable information (PII), including Aadhaar-linked data, shall be encrypted at rest and in transit.
- The system shall enforce role-based access control (RBAC) so that staff and administrator functions are inaccessible to citizen accounts.
- The system shall maintain an audit log of all administrative configuration changes and manual queue overrides.
- The system shall comply with applicable data protection regulations (e.g., India's Digital Personal Data Protection Act) regarding consent, storage, and deletion of user data.

5.4 Software Quality Attributes

- Usability: A first-time citizen user shall be able to complete a booking in under 3 minutes without external assistance.
- Reliability: The system shall gracefully degrade (e.g., fall back to static token issuance) if the ML prediction service is unavailable.
- Scalability: The architecture shall allow horizontal scaling of the backend and database independently of the frontend.
- Maintainability: The ML models shall be retrainable and redeployable without requiring changes to the backend API contract.
- Portability: The mobile application shall be deployable to both Android and iOS from a single Flutter codebase.

5.5 Business Rules

- Only verified staff or administrators may reclassify a user's priority category.
- A branch administrator may configure local priority weightings, but may not disable priority scheduling entirely for legally mandated categories.
- No-show appointments beyond the configured grace period forfeit the reserved slot but may be automatically rebooked into the next available slot at user request.
- Feedback ratings below the configured threshold shall trigger a review workflow rather than automatic staff penalty.

6. Other Requirements

6.1 Database Requirements

The system requires, at minimum, the following core entities: Users (user_id, name, phone, email, category, role), Appointments (appointment_id, user_id, service_type, branch_id, slot_time, status), Branches (branch_id, branch_name, location, current_crowd), and Queue (queue_id, appointment_id, position, estimated_wait_time). Additional supporting tables shall include Feedback, Complaints, Notifications, and ML Prediction Logs to support Sections 4.12–4.13 and Section 4.3.

6.2 Internationalization Requirements

All user-facing text shall be externalized into resource files to support the multilingual capability described in Section 4.14 without requiring code changes to add a new language.

6.3 Legal and Compliance Requirements

Where Aadhaar-based verification is used, the system shall operate in accordance with UIDAI guidelines and shall not retain Aadhaar numbers beyond what is legally permitted for verification purposes.

Appendix A: Glossary

Term	Definition
SRS	Software Requirements Specification
ML	Machine Learning
OTP	One-Time Password
JWT	JSON Web Token, used for authentication
IVR	Interactive Voice Response, an automated telephone system
RBAC	Role-Based Access Control
SLA	Service Level Agreement
No-show	A booked appointment where the citizen fails to check in within the grace period
UIDAI	Unique Identification Authority of India, the body governing Aadhaar

Appendix B: Analysis Models

B.1 High-Level System Architecture

Citizens interact with the Web/Mobile application, which communicates with the FastAPI Backend Server. The backend server integrates three core subsystems: the Database (PostgreSQL), the ML Engine (wait-time prediction, no-show prediction, and demand forecasting), and the Map Module (branch location and crowd visualization, via OpenStreetMap/Leaflet.js). The Admin Dashboard, used by managers and staff, consumes the same backend APIs to monitor and control operations.

Architecture layers, top to bottom:

- Citizens → Web / Mobile Application (React + Vite + Tailwind; Flutter)
- Web / Mobile Application → FastAPI Backend Server (REST + WebSocket API)
- FastAPI Backend Server → Database | ML Engine | Map Module (parallel integrations)
- FastAPI Backend Server → Admin Dashboard (Managers & Employees)

B.2 Working Flow

- User Login
- Choose Service
- System Predicts Duration (ML Engine)
- Slot Allocated
- User Receives Confirmation
- Queue Monitored Live
- Dynamic Optimization (as needed)
- Service Completion → Feedback Prompt

B.3 Recommended Technology Stack

Layer	Technology
Web Frontend	React.js, Vite, Tailwind CSS, ShadCN UI
Mobile App	Flutter (Android + iOS, single codebase)
Backend	FastAPI (Python); Django as an alternative
Database	PostgreSQL (Supabase); MySQL as an alternative
Machine Learning	Scikit-learn, XGBoost, Prophet, Pandas, NumPy
Maps & Location	OpenStreetMap + Leaflet.js
Authentication	JWT (FastAPI-JWT-Auth, PyJWT)
Notifications	Firebase Cloud Messaging, SMTP/Gmail API, SMS Gateway
Real-Time Updates	WebSockets (FastAPI WebSockets)
Hosting	Vercel (frontend), Render/Railway/Koyeb (backend), Supabase (database)

Appendix C: To Be Determined List

#	TBD Item	Section Reference
TBD-1	Final list of pilot service center categories and their specific service catalogs.	1.4 / 2.2
TBD-2	Exact SLA windows for complaint escalation per organization.	4.13
TBD-3	Full list of supported regional languages beyond the initial launch language.	4.14
TBD-4	Specific data retention period for Aadhaar-linked verification data.	6.3
TBD-5	Finalized concurrent-user and throughput targets per organization size.	5.1